

Earley’s Recognizer: Implementation and Complexity Analysis in Isabelle/HOL[★]

Tobias Nipkow¹  and Alexander Haberl²

¹ Technical University of Munich, Germany <https://www.proof.cit.tum.de/~nipkow>

² Technical University of Munich, Germany alexander.haberl@tum.de

Abstract. This paper starts from a recent formal verification of an abstract inductive but not directly executable (i.e. recursive) version of Earley’s recognizer. The aim of that work was a particularly simple, abstract formalization and stepwise refinement towards executability. We continue this work and refine it to an executable implementation together with a proof of the running time: cubic if the n -th element of a list can be accessed in constant time, and quartic if access takes linear time. We present the running time analysis with the help of a simple Big O formalization. Everything has been formalized in Isabelle/HOL.

1 Introduction

Earley’s recognizer [4,5] is a famous algorithm for deciding the word problem for all context-free grammars. It runs in cubic time in general and in quadratic time for unambiguous grammars. A parser would return one or more parse trees as well. In this paper we restrict attention to the recognizer. Earley’s recognizer or parser are frequently covered in the literature (e.g. [1]) and are the basis of many further refinements (e.g. [20]). This paper focuses on Earley’s original version.

The first rigorous mathematical proof of Earley’s algorithm is due to Jones [8,7]. This was the starting point of the work by Nipkow and Rau [17]. We continue their work as follows. First we refine the recognizer from sets to lists, then from inductive to recursive, and finally we augment the lists with a more efficient array-like data structure. Then we present a complexity analysis that proves the well-known cubic bound if we can access the n -th element of a list in constant time. If that access takes linear time, we obtain a quartic bound. Like our predecessors, we assume that the grammar is epsilon-free.

Earley’s algorithm supports parsing as well, but the extension is non-trivial, at least for non-ambiguous grammars, and Earley’s original algorithm could return spurious parse trees [21]. Scott [20, Section 1] gives a short history of the generalizations of Earley’s recognizer to a parser. Part of the difficulty is how to store multiple parse trees efficiently. Rau and Nipkow [18] have verified an Earley parser based on the work of Scott. Their work is more monolithic whereas ours aims for simple refinement steps and proofs.

This paper is largely based on the master’s thesis by the second author [6].

[★] Dedicated to Lawrence C. Paulson on the occasion of his 70th birthday

There are a number of related publications on verified parsing algorithms and parser generators for a variety of grammars and formalisms: for SLR grammars [2], parsing expression grammars [10,3], parser combinators [19], LL(1) grammars [13] and non-left-recursive grammars [14]. There is also a verified validator for LR(1) automata [9]. What sets our work apart is the algorithm and the complexity analysis.

2 Isabelle Notation

The notation $t :: \tau$ means that term t has type τ . Basic types include *bool* and *nat*, while type variables are written $'a$, $'b$ etc. Most type constructors are written postfix, such as $'a \text{ set}$ and $'a \text{ list}$, and the function space arrow is $\Rightarrow$. The image of function f over set M is denoted by $f ` A = \{b \mid \exists a \in A. f a = b\}$.

Functions *fst* and *snd* return the first and second components of a pair.

Lists are constructed from the empty list $[]$ using the infix cons-operator $(\#)$. The operator $(@)$ appends two lists, $|xs|$ denotes the length of xs , *set* converts a list to a set, $xs ! n$ returns the n -th element of the list xs (starting with $n = 0$) and $xs[n := x]$ is xs with $xs ! n$ replaced by x .

Algebraic data types are defined using the **datatype** keyword. The *option* type is predefined

datatype $'a \text{ option} = \text{None} \mid \text{Some } 'a$

with the function *the* $(\text{Some } x) = x$.

The notation $\llbracket A_1, \dots, A_n \rrbracket \Longrightarrow B$ denotes an implication with premises $A_1, \dots, A_n$ and conclusion B . Sometimes we use the inference rule presentation

$$\frac{A_1 \quad \dots \quad A_n}{B}$$

Equality on type *bool* denotes logical equivalence.

2.1 Context-Free Grammars

All our types are parameterized by type variables $'n$ and $'t$, the types of **non-terminals** and **terminals**. **Symbols** are a tagged union type:

datatype $('n, 't) \text{ sym} = \text{Nt } 'n \mid \text{Tm } 't$

Variable convention: $A, B, C :: 'n$. For brevity we sometimes drop the $'n$ and $'t$ parameters, e.g. we write *sym* instead of $('n, 't) \text{ sym}$.

A **production**, informally written $A \rightarrow \alpha$, is a pair of $A :: 'n$ (the *lhs*) and $\alpha :: \text{sym list}$ (the *rhs*).

Our work is primarily based on sets of productions rather than grammars: the start symbol is irrelevant most of the time. *For succinctness, we use **grammar** to refer to a set (or list) of productions.* Moreover, we only restrict to finite sets of productions when necessary.

The identifier P is reserved for sets of productions. Every P induces a single step derivation relation on *sym* lists in the standard manner:

$$(A, \beta) \in P \implies P \vdash \alpha @ [Nt\ A] @ \gamma \Rightarrow \alpha @ \beta @ \gamma$$

Its transitive-reflexive closure is denoted by $P \vdash \alpha \Rightarrow^* \beta$.

The language of some nonterminal A generated by P is easily defined:

$$Lang\ P\ A = \{w \mid P \vdash [Nt\ A] \Rightarrow^* map\ Tm\ w\}$$

3 The Story So Far

In this section we summarize the contents of our starting point, the work by Nipkow and Rau [17]: a refinement of an initial inductive definition, via the standard definition to something resembling a worklist algorithm, but still as an inductive definition.

3.1 Inductive Definition of Earley's Item Sets

An Earley recognizer decides if some word is in the language described by a grammar by generating a set of so-called *items* (also called *states* in the literature) and checking if it contains a *final* item. In the sequel, we fix the following “global variables”:

- a grammar P
- a start symbol S
- an input $w :: syms\ list$ consisting only of terminal symbols.

An item is a triple *Item* $r\ d\ i$ where $r = (A, \alpha)$ is a production from P , d is a natural number such that $d \leq |\alpha|$, indicating how far the algorithm has recognized α , and i is a natural number representing the start index of the subword of w recognized by this item. The three projection functions are *prod*, *dot* and *from*. We use the letters x and y for items.

A production $A \rightarrow \alpha$ together with a *dot* d is shown as $A \rightarrow \alpha_1 \bullet \alpha_2$, where α_1 is the prefix of the first d symbols of α and α_2 is the suffix. We will informally show items as pairs $(A \rightarrow \alpha \bullet \beta, i)$.

Below we need the following auxiliary functions on items that query or modify the dotted production but not the *from* component of the item.

mv_dot moves the dot one position to the right:

$$mv_dot\ (A \rightarrow \alpha \bullet s \beta, i) = (A \rightarrow \alpha s \bullet \beta, i)$$

is_complete x checks if x is of the form $(A \rightarrow \alpha \bullet, i)$.

next_sym $x\ s$ is true iff x is of the form $(A \rightarrow \alpha \bullet s \beta, i)$.

Note that *next_sym* $x\ s$ implies $\neg is_complete\ x$.

Next we give an abstract inductive definition of *Earley*, a set of pairs (j, x) of an index and an item. However, it is more intuitive to present it as an inductive definition of indexed item sets: we write $x \in \mathcal{E}\ j$ instead of $(j, x) \in Earley$. In the indexed presentation, we define item sets $\mathcal{E}\ j, j \leq |w|$, that depend on each other.

However, the dependence is in one direction only: later sets depend on earlier ones, not the other way around. Thus the $\mathcal{E} j$ can be generated in ascending order. This is the subject of the next section but plays no role now.

The intuitive meaning of $Item\ r\ d\ i \in \mathcal{E} j$ is that the subword *slice* $i\ j\ w$ (which stands for *drop* i (*take* $j\ w$)) has been recognized. We call i the *start index* and j the *end index* (which is excluded).

The four defining rules are: one rule for the initial set of items, and one rule for each of the core operations that expand the set of items: scanning, prediction, and completion.

$$\text{INIT: } \frac{x \in \text{Init}}{x \in \mathcal{E} 0} \quad \text{where } \text{Init} = \{Item\ r\ 0\ 0 \mid r \in P \wedge lhs\ r = S\}$$

$$\text{SCAN: } \frac{x \in \mathcal{E} j \quad j < |w| \quad next_sym\ x\ (w\ !\ j)}{mv_dot\ x \in \mathcal{E} (j + 1)}$$

$$\text{PREDICT: } \frac{x \in \mathcal{E} k \quad x' \in Predict\ k\ x}{x' \in \mathcal{E} k}$$

$$\text{where } Predict\ k\ x = \{Item\ r\ 0\ k \mid r \in P \wedge next_sym\ x\ (Nt\ (lhs\ r))\}$$

$$\text{COMPLETE: } \frac{\begin{array}{c} y \in \mathcal{E} k \quad is_complete\ y \\ x \in \mathcal{E} (from\ y) \quad next_sym\ x\ (Nt\ (lhs\ (prod\ y))) \end{array}}{mv_dot\ x \in \mathcal{E} k}$$

Input w is *accepted* if $\mathcal{E} |w|$ contains a final item ($S \rightarrow \gamma\bullet, 0$), i.e. the entire input has been recognized:

$$is_final\ x = (lhs\ (prod\ x) = S \wedge from\ x = 0 \wedge is_complete\ x)$$

$$accepted = (\exists x \in \mathcal{E} |w|. is_final\ x)$$

We proved *soundness* and *completeness* of the item-based acceptance w.r.t. the grammar:

$$accepted \implies P \vdash [Nt\ S] \Rightarrow^* w \quad \text{and} \quad P \vdash [Nt\ S] \Rightarrow^* w \implies accepted$$

3.2 The Standard Definition

The standard definition of Earley's algorithm generates each $\mathcal{E} j$ completely before moving on to $\mathcal{E} (j + 1)$. This sequential strategy works because elements of $\mathcal{E} j$ do not depend on $\mathcal{E} k$ where $j < k$. Therefore Earley [5] starts with a more algorithmic formulation. The sets $\mathcal{E} 0, \mathcal{E} 1, \dots, \mathcal{E} |w|$ are generated in that order. We follow Jones and define a sequence of *bins* (= item sets), where the i -th bin is meant to hold $\mathcal{E} i$. Function *bins* recursively computes the list of bins

$$\begin{aligned} bins\ 0 &= [Close\ []\ Init] \\ bins\ (k + 1) &= (let\ Bs = bins\ k\ in\ Bs\ @\ [Close\ Bs\ (Scan\ k\ (last\ Bs))]) \end{aligned}$$

In each step, it takes the (currently) last bin, generates a set of new items by scanning

$$Scan\ k\ B = \{mv_dot\ x \mid x \in B \wedge next_sym\ x\ (w\ !\ k)\}$$

and closes this set under prediction and completion. The closure operator *Close* takes two arguments: the current list of bins and the result of scanning. It is defined in analogy with PREDICT and COMPLETE:

$$\frac{x \in B}{x \in Close\ Bs\ B} \quad \frac{x \in Close\ Bs\ B \quad x' \in Predict\ |Bs|\ x}{x' \in Close\ Bs\ B}$$

$$\frac{is_complete\ y \quad \frac{y \in Close\ Bs\ B \quad x' \in Complete\ (Bs\ @\ [Close\ Bs\ B])\ y}{x' \in Close\ Bs\ B}}{x' \in Close\ Bs\ B}$$

$$Complete\ Bs\ y = mv_dot\ ' \{x \in Bs\ !\ from\ y \mid next_sym\ x\ (Nt\ (lhs\ (prod\ y)))\}$$

In the end we proved the following correctness theorem:

$$i \leq |w| \implies bins\ |w|\ !\ i = \mathcal{E}\ i$$

3.3 Epsilon-free Closure

To simplify the computation of *Close*, we follow Jones and assume that *P* is epsilon-free in the rest of the paper, i.e. there is no production in *P* with an empty *rhs*. In that case, in the completion rule for *Close*, $x \in (Bs\ @\ [Close\ Bs\ B])\ !\ from\ y$ can be simplified to $x \in Bs\ !\ from\ y$. Formally, we define a variant *Close1* of *Close* with the simplified completion rule and prove their equivalence:

$$\llbracket wf_bins1\ Bs; wf_bin1\ |Bs|\ B \rrbracket \implies Close1\ Bs\ B = Close\ Bs\ B$$

We mostly ignore wellformedness conditions in this paper but the crucial point of $wf_bin1\ |Bs|\ B$ is that all complete items in *B* have a *from* index of $< |Bs|$. Thus completion only needs to consult *Bs* and not the current bin *B*. Moreover $wf_bins1\ Bs \equiv \forall k < |Bs|. wf_bin1\ k\ (Bs\ !\ k)$.

3.4 One-Pass Closure

Our previous work concludes with an abstract one-pass formulation of *Close1*. It is still on the level of sets and intentionally nondeterministic.

We formulate the one-pass closure as a transition system $Bs \vdash (B, C) \rightarrow (B', C')$ where *B*, *C*, *B'*, *C'* are sets of items: *B* is the current set whose closure is to be computed (the “worklist”), *C* is the accumulator for the closure, and (B', C') is the result of a) moving some item $x \in B$ to the accumulator (i.e. $C' = C \cup \{x\}$ and b) extending the worklist with all results of prediction or completion (depending on *x*). The definition is again inductive:

$$\frac{x \in B \quad \neg \text{is_complete } x}{Bs \vdash (B, C) \rightarrow (B \cup \text{Predict } |Bs| \ x - (C \cup \{x\}), \{x\} \cup C)}$$

$$\frac{x \in B \quad \text{is_complete } x}{Bs \vdash (B, C) \rightarrow (B \cup \text{Complete } Bs \ x - (C \cup \{x\}), C \cup \{x\})}$$

Note that a more efficient version of $B \cup \text{Predict } |Bs| \ x - (C \cup \{x\})$ would be $B - \{x\} \cup (\text{Predict } |Bs| \ x - (C \cup \{x\}))$ (and similarly for **Complete**) because B and C should be disjoint. For simplicity we have ignored this optimization.

The full closure “algorithm” consists of the stepwise reduction of B to the empty set:

$$\text{close2 } Bs \ B = (\text{SOME } C. Bs \vdash (B, \emptyset) \rightarrow^* (\emptyset, C))$$

where *SOME* is Hilbert’s epsilon operator: *SOME* $x. Q$ denotes an arbitrary (but fixed!) x that satisfies Q (if such an x exists). In our case it does exist, as witnessed by the following lemma:

$$\llbracket \text{wf_bins1 } Bs; \text{wf_bin1 } |Bs| \ B \rrbracket \implies \exists C'. Bs \vdash (B, C) \rightarrow^* (\emptyset, C')$$

The proof is by induction on a suitable wellfounded relation (that is based on the fact that there are only finitely many wellformed items). Although it is not obvious, there is a unique C' such that $Bs \vdash (B, C) \rightarrow^* (\emptyset, C')$, i.e. the result of *close2* is independent of which result *SOME* chooses.

As for *Close* and *Close1*, we can also prove the equivalence of *Close1* and *close2*. This is relegated to Section 3.5.

Of course we still need to show termination of $\rightarrow$. This follows because the set of well-formed items (let us call it I) is finite, because there are only finitely many dotted productions and *from* fields. In each $(B, C) \rightarrow (B', C')$ step, either $|I - (B \cup C)|$ decreases or it remains the same (because no new item was generated) and $B' \subset B$, which implies termination. By induction on this wellfounded relation it follows that eventually $B = \emptyset$ because if not, either prediction or completion applies.

Finally we can replace *Close* by *close2* in the definition of *bins* and obtain (by proof)

$$\begin{aligned} \text{bins } 0 &= [\text{close2 } \llbracket \text{Init} \rrbracket] \\ \text{bins } (k + 1) &= (\text{let } Bs = \text{bins } k \text{ in } Bs @ [\text{close2 } Bs \ (\text{Scan } k \ (\text{last } Bs))]) \end{aligned}$$

Of course, we have not yet arrived at an executable formulation because of the presence of *SOME* in *close2*.

3.5 Equivalence of *close2* and *Close1*

We will sketch the proof of

$$\llbracket \text{wf_bins1 } Bs; \text{wf_bin1 } |Bs| \ B \rrbracket \implies \text{close2 } Bs \ B = \text{Close1 } Bs \ B$$

because it was omitted in [17]. All proofs are by obvious inductions.

The following lemmas are generally useful:

$$\llbracket Bs \vdash (B, C) \rightarrow^* (B', C'); B \cap C = \emptyset \rrbracket \implies B' \cap C' = \emptyset$$

$$Bs \vdash (B, C) \rightarrow^* (B', C') \implies B \cup C \subseteq B' \cup C'$$

Soundness $(Bs \vdash (B, \emptyset) \rightarrow^* (\emptyset, C) \implies C \subseteq \text{Close1 } Bs B)$ follows in two obvious steps:

$$Bs \vdash (B, C) \rightarrow (B', C') \implies B' \cup C' \subseteq \text{Close1 } Bs (B \cup C)$$

$$Bs \vdash (B, C) \rightarrow^* (B', C') \implies B' \cup C' \subseteq \text{Close1 } Bs (B \cup C)$$

Completeness is a bit trickier. The key lemma says that in an $\rightarrow$ sequence, where an element x starts in B and ends up in C' , there is a $\rightarrow$ step where that transition takes place.

$$\begin{aligned} & \llbracket x \notin C; B \cap C = \emptyset; Bs \vdash (B, C) \rightarrow^* (B', C'); x \in C' \rrbracket \\ & \implies \exists B_1 \ C_1 \ B_2 \ C_2. \\ & \quad Bs \vdash (B, C) \rightarrow^* (B_1, C_1) \wedge \\ & \quad x \in B_1 \wedge \\ & \quad Bs \vdash (B_1, C_1) \rightarrow (B_2, C_2) \wedge x \notin B_2 \wedge Bs \vdash (B_2, C_2) \rightarrow^* (B', C') \end{aligned}$$

Completeness can be proved from this lemma.

$$\llbracket x \in \text{Close1 } Bs B; Bs \vdash (B, \emptyset) \rightarrow^* (\emptyset, C) \rrbracket \implies x \in C$$

Altogether this yields the desired

$$Bs \vdash (B, \emptyset) \rightarrow^* (\emptyset, C) \implies C = \text{Close1 } Bs B$$

4 Refinement

This section is concerned with an efficient, executable implementation of the closure operation on bins, and thus all of the recognizer. We proceed in three steps:

1. Implement sets by lists.
2. Turn the inductive definition into a (functional) while-loop.
3. Augment bins with a data structure for efficient search for items.

4.1 From Sets to Lists

This is a canonical data refinement step. We take the worklist algorithm for closure from Section 3.4 and replace *set* by *list*. The new transition relation has the syntax $Bs \vdash_L (B, C) \rightarrow (B', C')$ where B, C, B', C' are item lists and Bs is a list of item lists:

$$\begin{aligned} & \frac{\neg \text{is_complete } x}{Bs \vdash_L (x \# B, C) \rightarrow (\text{Predict}_L |Bs| \ x \cup_L B -_L (x \#_L C), x \#_L C)} \\ & \frac{\text{is_complete } x}{Bs \vdash_L (x \# B, C) \rightarrow (\text{Complete}_L Bs \ x \cup_L B -_L (x \#_L C), x \#_L C)} \end{aligned}$$

where

$$\begin{aligned} \text{Predict}_L k x &= \\ \text{map } (\lambda p. \text{Item } p \ 0 \ k) &(\text{filter } (\lambda p. \text{next_sym } x \ (\text{Nt } (\text{lhs } p)))) \ ps) \\ \text{Complete}_L Bs y &= \\ \text{map } \text{mv_dot} &(\text{filter } (\lambda x. \text{next_sym } x \ (\text{Nt } (\text{lhs } (\text{prod } y)))) \ (Bs \ ! \ \text{from } y)) \end{aligned}$$

The binary operations $(\#_L)$, $(\cup_L)$ and $(-_L)$ implement the corresponding set operations on lists. They are defined in Appendix A. We have also replaced the set of productions P by a list ps and assume $P = \text{set } ps$.

Correctness and termination can be proved straightforwardly:

$$\begin{aligned} Bs \vdash_L (B, C) \rightarrow^* (B', C') &\implies \\ \text{map set } Bs \vdash (\text{set } B, \text{set } C) &\rightarrow^* (\text{set } B', \text{set } C') \\ \exists C'. Bs \vdash_L (B, C) \rightarrow^* &([], C') \end{aligned}$$

assuming $\text{wf_bins1 } (\text{map set } Bs)$ and $\text{wf_bin1 } |Bs| \ (\text{set } B)$.

4.2 From Inductive to Recursive

Now we take the step from inductive predicate to directly executable functional algorithm. We employ an Isabelle/HOL library theory defining a while-combinator

$$\text{while_option} :: ('a \Rightarrow \text{bool}) \Rightarrow ('a \Rightarrow 'a) \Rightarrow 'a \Rightarrow 'a \ \text{option}$$

whose definition is explained elsewhere [15]. All we need to know is that it is executable by means of this recursion equation provable as a lemma:

$$\text{while_option } b \ c \ s = (\text{if } b \ s \ \text{then } \text{while_option } b \ c \ (c \ s) \ \text{else } \text{Some } s) \quad (1)$$

Termination is equivalent with a *Some* result.

The closure computation is performed by the obvious worklist loop:

$$\begin{aligned} \text{close2}_L Bs &= \text{while_option } (\lambda(B, C). B \neq []) \ (\text{step2}_L Bs) \\ \text{step2}_L Bs \ (x \ \# \ B, C) &= \\ (\text{let } \text{nexts} &= \text{if } \text{is_complete } x \ \text{then } \text{Complete}_L Bs \ x \ \text{else } \text{Predict}_L |Bs| \ x \\ \text{in } (\text{nexts} \cup_L B &-_L (x \ \#_L C), x \ \#_L C)) \end{aligned}$$

By the same well-founded induction as before we can show termination of close2_L :

$$\exists C'. \text{close2}_L Bs \ (B, []) = \text{Some } ([], C')$$

Correctness is provable by a standard invariant preservation argument:

$$\begin{aligned} &\llbracket \text{wf_bins1 } ps \ w_0 \ (\text{map set } Bs); \text{wf_bin1 } ps \ w_0 \ |Bs| \ (\text{set } B); \\ &\text{close2}_L Bs \ (B, []) = \text{Some } ([], C) \rrbracket \\ \implies &\text{map set } Bs \vdash (\text{set } B, \emptyset) \rightarrow^* (\emptyset, \text{set } C) \end{aligned}$$

4.3 Efficient Access to Bins

The expensive part of the closure construction is checking, in each step, if a newly generated item is already present (in B or C). The number of possible items is bounded by the number of dotted productions (a constant D depending on P) times the number of possible *from* values, i.e. $D * |w|$. We will store B and C not just as linear lists but will pair them with a version partitioned by *from* values, i.e. as a list of lists indexed by *from* values. When searching for an item x , if we can access partition *from* x directly, the search takes time at most D , the maximum size of each partition. When processing the i -th input character, i.e. $i = |Bs|$, we pair B (and C) with a list (think array) F of size i such that $F ! j$ is a list of all $x \in B$ where *from* $x = j$. The corresponding data type:

datatype ('n, 't) *item_list* = *IL* (('n, 't) *item list*) (('n, 't) *item list list*)

with the projection functions *list* and *froms*. The invariant

$$\begin{aligned} \text{inv}_{IL} (IL \ xs \ fs) = & \\ & ((\forall x \in \text{set } xs. \text{from } x < |fs|) \wedge \\ & (\forall i < |fs|. \text{set } (fs ! i) = \{x \in \text{set } xs \mid \text{from } x = i\}) \wedge \\ & (\forall i < |fs|. \text{distinct } (fs ! i)) \wedge \text{distinct } xs) \end{aligned}$$

ensures that fs is long enough to accommodate all of xs , that fs is an indexed version of xs , and that there are no duplicates.

This is a data refinement step where we replace item lists by their indexed version ('n, 't) *item_list*. The refinement of the closure algorithm *close2_L* looks like this:

$$\begin{aligned} \text{step2}_{IL} \ Bs \ (IL \ (x \# \ xs) \ fs, \ C) = & \\ & (\text{let } \text{nexts} = \text{if } \text{is_complete } x \text{ then } \text{Complete}_L \ Bs \ x \text{ else } \text{Predict}_L \ |Bs| \ x \\ & \text{in } (\text{nexts} \cup_{IL} \ IL \ (x \# \ xs) \ fs -_{IL} \ (x \#_{IL} \ C), \ x \#_{IL} \ C)) \\ \text{close2}_{IL} \ Bs \ B = & \\ \text{list } (\text{snd } (\text{the } (\text{while_option } (\lambda(ilB, ilC). \text{list } ilB \neq []) \ (\text{step2}_{IL} \ Bs) \\ & (B, \text{empty}_{IL} \ |Bs|)))) \end{aligned}$$

Upon termination of *while_option*, *list* $\circ$ *snd* $\circ$ *the* extracts the closure as an item list. The set operations *empty_{IL}*, (*#_{IL}*), (*∪_{IL}*) and (*−_{IL}*) are defined in Appendix B.

Finally, we are ready for the executable top-level algorithm, a refinement of *bins* from Section 3.2:

$$\begin{aligned} \text{bins}_{IL} \ 0 &= [\text{close2}_{IL} [] \ (IL_list \ 0 \ \text{Init}_L)] \\ \text{bins}_{IL} \ (k + 1) &= \\ & (\text{let } Bs = \text{bins}_{IL} \ k \text{ in } Bs \ @ \ [\text{close2}_{IL} \ Bs \ (IL_list \ |Bs| \ (\text{Scan}_L \ k \ (\text{last } Bs))])) \\ \text{Init}_L &= \text{map } (\lambda p. \text{Item } p \ 0 \ 0) \ (\text{filter } (\lambda p. \text{lhs } p = S) \ ps) \\ \text{Scan}_L \ k \ Bs &= (\text{let } s = w ! k \text{ in } \text{map } \text{mv_dot} \ (\text{filter } (\lambda b. \text{next_sym } b \ s) \ Bs)) \end{aligned}$$

Note that at the end $close2_{IL}$ throws away the *froms* component and returns only the *list* which $bins_{IL}$ adds to Bs . The *froms* part is not useful any longer. (Except for the final search for an *is_final* item, but the improvement of that single step is marginal.)

Correctness is expressed as in Section 3.2:

$$i \leq |w| \implies \text{set } (bins_{IL} |w| ! i) = \mathcal{E} \ i$$

5 Complexity

We have analyzed the running time of the recognizer following the approach detailed elsewhere [16]: from the definition of an executable function $f :: \tau \Rightarrow \tau'$, we automatically generate a (time) function definition $T_f :: \tau \Rightarrow nat$ such that $T_f x$ is the number of computation steps of $f x$ where we count only calls of recursive user-defined functions. For example, the time function T_{isin_list} (for *isin_list* from Appendix A) is defined like this:

$$\begin{aligned} T_{isin_list} [] a &= 1 \\ T_{isin_list} (x \# xs) a &= (\text{if } x = a \text{ then } 0 \text{ else } T_{isin_list} xs a) + 1 \end{aligned}$$

The boon and bane of this approach is that one obtains very detailed formulas with somewhat arbitrary constants. Arbitrary because they depend on the abstract machine model embodied in the generation of the time functions, which may be quite different from a real machine. As an extreme example, consider this upper bound:

$$\begin{aligned} & \llbracket list \ il_1 \neq []; wf_bins1 \ ps \ w_0 \ (map \ set \ Bs); \forall i < |Bs|. \ distinct \ (Bs \ ! \ i); \\ & wf_bin1 \ ps \ w_0 \ |Bs| \ (set_{IL} \ il_1); inv_{IL} \ il_1; |froms \ il_1| = |Bs| + 1; \\ & wf_bin1 \ ps \ w_0 \ |Bs| \ (set_{IL} \ il_2); inv_{IL} \ il_2; |froms \ il_2| = |Bs| + 1 \rrbracket \\ & \implies T_{step2_{IL}} \ Bs \ (il_1, il_2) \\ & \leq L * (K + 1) * (|Bs| + 1) * \\ & \quad (7 * T_{nth_{IL}} |Bs| + 3 * L * (K + 1) + 7 + 2 * (K + 2)) + \\ & \quad 7 * T_{nth_{IL}} |Bs| + 3 * (|Bs| + 1) + 2 * L * (K + 1) + 7 + (K + 1) \end{aligned}$$

Note the following:

- A large number of well-formedness assumptions are required, which we will elide in the rest of this section.
- In the upper bound term, K and L are constants depending on ps .
- Function $T_{nth_{IL}}$ is a parameter of the analysis and assumed to be an upper bound of the real T_{nth} , and also of T_{list_update} , the time function for the list update function $xs[i := x]$.

There are two interesting choices for $T_{nth_{IL}}$:

- Linear time is the standard situation in functional programs.
- Constant time models array-like access and update of lists. We could achieve this by another refinement step using Lammich's and Lochbihler's Collections library [11,12] that offers imperative implementations of arrays with a purely functional, list-like interface specification.

Both choices are discussed below.

5.1 Interlude: Asymptotic Complexity

In order to avoid such constant-studded upper bounds as shown above for $T_{step2_{IL}}$ we introduce a simple big O framework, just enough for our purposes. In the following, f and g are of type $'a \Rightarrow nat$ and Q of type $'a \Rightarrow bool$. As an analogue of $f \in O(g)$ we define

$$(f \leq_O g \text{ IF } Q) = (\exists c \ d. \forall x. Q \ x \longrightarrow f \ x \leq c * g \ x + d)$$

The $IF \ Q$ is dropped if Q is *True* everywhere.

In contrast to $f \in O(g)$, f and g are not defined on numbers but on an arbitrary type $'a$. For example, $(\lambda xs. 2 * |xs| + 3) \leq (\lambda xs. |xs|^2)$ is true: take $c = 1$ and $d = 4$. Our notation combines the measure function on some domain (here: the length of a list) with the growth function (here: linear and quadratic). It is easy to show that $f \circ m \leq_O g \circ m$, where $m :: 'a \Rightarrow nat$, implies $f \in O(g)$ under mild assumption (for example, that m is surjective and g is 0 only for finitely many arguments).

In concrete instances, f , g and Q will be λ abstractions over the same parameter(s). Therefore we introduce a quantifier O that shares the parameters:

$$(O \ x. Q \ x \longrightarrow f \ x \leq g \ x) = (f \leq_O g \text{ IF } Q)$$

For example, the fictitious

$$(\lambda(xs, i). T_f \ xs \ i) \leq_O (\lambda(xs, i). i) \text{ IF } \lambda(xs, i). i < |xs|$$

becomes the more compact $O \ (xs, i). i < |xs| \longrightarrow T_f \ xs \ i \leq i$

5.2 Earley Complexity

We have analyzed the complexity of our Earley recognizer with all constants present and in terms of an upper bound $T_{nth_{IL}}$ for T_{nth} . We present the result at the end of this section. For readability the running times of selected auxiliary functions are presented in O notation. Except in the final result, *we assume $T_{nth_{IL}}$ is constant and we drop all preconditions for readability.*

Because of the constant time (!) assumption, single element set operations are constant time as well

$$O \ (il, x). T_isin_{IL} \ il \ x \leq 1$$

$$O \ (x, il). x \ T_#_{IL} \ il \leq 1$$

while combinations of sets take linear time:

$$O \ (xs, il). T_cup_{IL} \ xs \ il \leq |xs|$$

$$O \ (il_1, il_2). T_minus_{IL} \ il_1 \ il_2 \leq |list \ il_1| + |froms \ il_1|$$

Each evaluation of *step2_{IL}* requires a (constant) number of set operations that are all of either constant time or linear time in $|Bs|$ (because the number of well-formed items in the bin under construction is linear in $|Bs|$). Moreover, *Predict_L* is obviously constant time (because *ps* is fixed). Evaluation of *Complete_L* $Bs \ x$ is linear in $Bs ! from \ x$, which is linear in $|Bs|$. Hence the overall complexity of *step2_{IL}* is also linear in $|Bs|$:

$$O(Bs, il_1, il_2). T_step2_{IL} Bs (il_1, il_2) \leq |Bs|$$

Now we need to analyze the complexity of $steps2_{IL}$, which is defined by means of $while_option$. However, the derived computation rule (1) for $while_option$ is not total (in contrast to the non-executable definition). Thus it cannot be used as the basis for the definition of a running time function. Therefore we have to set up T_while_option by hand. We simply add an accumulator that counts the computation steps:

$$T_while_option2\ b\ f\ tf = \\ while_option\ (\lambda(x, n). b\ x)\ (\lambda(x, n). (f\ x, n + tf\ x))$$

Note that for simplicity we assume that the test b is for free, which it is in our application. In the end we project on the counter:

$$T_while_option\ b\ f\ tf\ x = snd\ (the\ (T_while_option2\ b\ f\ tf\ (x, 0)))$$

Now we can define the time function for $steps2_{IL}$:

$$T_steps2_{IL}\ Bs = \\ T_while_option\ (\lambda(il_1, il_2). list\ il_1 \neq [])\ (step2_{IL}\ Bs)\ (T_step2_{IL}\ Bs)$$

At the beginning of Section 4.3 we explained that there are at most $D * |w|$ well-formed items. More precisely, when processing input symbol number $|Bs|$, there are $D * (|Bs| + 1)$ well-formed items. Because each iteration of $steps2_{IL}\ Bs$ takes time proportional to $|Bs|$ and adds at least one item (or shifts one item from B to C), $steps2_{IL}$ (and thus also $close2_{IL}$) have quadratic complexity:

$$O(Bs, il_1, il_2). T_steps2_{IL} Bs (il_1, il_2) \leq |Bs|^2$$

Because each closure takes quadratic time and $bins_{IL}$ performs linearly many closure steps, its overall complexity is cubic:

$$O\ k. k \leq |w| \longrightarrow T_bins_{IL}\ k \leq k^3$$

Because *accepted* (see Section 3.1) requires a single scan of the last bin (to search for an *is_final* item), and its size is bounded by $D * |w|$, the time to compute *accepted* w is bounded by $T_bins_{IL}\ |w| + D * |w|$ and is thus cubic in $|w|$. This confirms Earley's informal analysis.

The above big O complexities are simplified versions of the more precise bounds we proved. We close this section with the upper bound we proved for our full recognizer. We spare the reader the definition of the constants:

$$C_1 * (|w| + 2)^3 + C_2 * (|w| + 2)^3 * T_nth_{IL}\ (|w| + 1)$$

Note that this bound is a function of T_nth_{IL} . It tells us that if function (!) takes linear time (as opposed to constant time as assumed above), our recognizer has worst-case quartic complexity. This confirms the informal analysis by Rau and Nipkow [18].

6 Conclusion

We have completed the development of a verified executable Earley recognizer started by Nipkow and Rau. In particular, we have verified its complexity. We have also verified an Earley parser for unambiguous grammars (with the same asymptotic complexity as the recognizer). We do not present it here because it does not raise any really new issues: the recognizer is simply extended with parse trees. Because the grammar is unambiguous, we do not have to worry about multiple parse trees for the same slice of the input. The initial Earley formalization summarized in “The Story So Far” is 1300 lines long, the extension presented in this paper, including the parser, runs to 4500 lines.

This is work in progress. The following points, in increasing order of difficulty, need to be addressed in the future:

- Lists indexed by *from* values should be implemented by proper arrays from the Isabelle Collections framework [11,12].
- The restriction to epsilon-free grammars should be lifted.
- The Big O framework needs to be automated. In particular, it should support deriving big O complexities directly, without having to derive precise bounds first.
- The parser (not presented in this paper) should be generalized from unambiguous to arbitrary grammars.

Acknowledgments. The first author would like to thank Larry Paulson for inventing the Isabelle theorem prover and for employing him many years ago to work on the further development of Isabelle. It changed his life.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Aho, A.V., Ullman, J.D.: The theory of parsing, translation, and compiling. Prentice-Hall, Inc. (1972)
2. Barthwal, A., Norrish, M.: Verified, executable parsing. In: Castagna, G. (ed.) Programming Languages and Systems, 18th European Symposium on Programming, ESOP 2009. Lecture Notes in Computer Science, vol. 5502, pp. 160–174. Springer (2009), https://doi.org/10.1007/978-3-642-00590-9_12
3. Blaudeau, C., Shankar, N.: A verified packrat parser interpreter for parsing expression grammars. In: Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs. p. 317. CPP 2020, Association for Computing Machinery, New York, NY, USA (2020), <https://doi.org/10.1145/3372885.3373836>
4. Earley, J.: An Efficient Context-Free Parsing Algorithm. Ph.D. thesis, Carnegie-Mellon University, Computer Science Department (1968)
5. Earley, J.: An efficient context-free parsing algorithm. Commun. ACM **13**(2), 94–102 (1970). <https://doi.org/10.1145/362007.362035>

6. Haberl, A.: Formal Verification of Earley’s Parsing Algorithm, Including Running Time Analysis. Master’s thesis, Department of Computer Science, Technical University of Munich (2026)
7. Jones, C.: Formal development of correct algorithms: An example based on Earley’s recogniser. Tech. Rep. T.R.12.095, IBM United Kingdom Laboratories, Hursley Park (Dec 1971), <http://homepages.cs.ncl.ac.uk/cliff.jones/publications/Other-TRs/TR12.095.pdf>
8. Jones, C.B.: Formal development of correct algorithms: An example based on Earley’s recogniser. In: Proceedings of ACM Conference on Proving Assertions About Programs. pp. 150–169. ACM (1972), <https://doi.org/10.1145/800235.807083>
9. Jourdan, J., Pottier, F., Leroy, X.: Validating LR(1) parsers. In: Seidl, H. (ed.) Programming Languages and Systems - ESOP 2012. pp. 397–416. Lecture Notes in Computer Science, Springer (2012), https://doi.org/10.1007/978-3-642-28869-2_20
10. Koprowski, A., Binsztok, H.: TRX: A formally verified parser interpreter. Log. Methods Comput. Sci. **7**(2) (2011), [https://doi.org/10.2168/LMCS-7\(2:18\)2011](https://doi.org/10.2168/LMCS-7(2:18)2011)
11. Lammich, P.: Collections framework. Archive of Formal Proofs (Nov 2009), <https://isa-afp.org/entries/Collections.html>, Formal proof development
12. Lammich, P., Lochbihler, A.: The Isabelle collections framework. In: Kaufmann, M., Paulson, L.C. (eds.) Interactive Theorem Proving, First International Conference, ITP 2010. vol. 6172, pp. 339–354. Springer (2010), https://doi.org/10.1007/978-3-642-14052-5_24
13. Lasser, S., Casinghino, C., Fisher, K., Roux, C.: A Verified LL(1) Parser Generator. In: Harrison, J., O’Leary, J., Tolmach, A. (eds.) 10th International Conference on Interactive Theorem Proving (ITP 2019). Leibniz International Proceedings in Informatics (LIPIcs), vol. 141, pp. 24:1–24:18. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2019), <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ITP.2019.24>
14. Lasser, S., Casinghino, C., Fisher, K., Roux, C.: Costar: a verified all(*) parser. In: Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation. p. 420434. PLDI 2021, Association for Computing Machinery (2021), <https://doi.org/10.1145/3453483.3454053>
15. Nipkow, T.: Verified efficient enumeration of plane graphs modulo isomorphism. In: van Eekelen, M.C.J.D., Geuvers, H., Schmaltz, J., Wiedijk, F. (eds.) Interactive Theorem Proving, ITP 2011. Lecture Notes in Computer Science, vol. 6898, pp. 281–296. Springer (2011), https://doi.org/10.1007/978-3-642-22863-6_21
16. Nipkow, T. (ed.): Functional Data Structures and Algorithms: A Proof Assistant Approach, ACM Books, vol. 67. ACM (2025), <https://dl.acm.org/doi/book/10.1145/3731369>, <https://fdsa-book.net>
17. Nipkow, T., Rau, M.: A note on proofs of Earley’s recognizer. In: Cavalcanti, A., Baxter, J. (eds.) The Practice of Formal Methods. Essays in Honour of Cliff Jones, Part II, Lecture Notes in Computer Science, vol. 14781, pp. 45–55. Springer (2024), https://link.springer.com/chapter/10.1007/978-3-031-66673-5_3
18. Rau, M., Nipkow, T.: A verified Earley parser. In: Bertot, Y., Kutsia, T., Norrish, M. (eds.) 15th International Conference on Interactive Theorem Proving, ITP 2024. LIPIcs, vol. 309, pp. 31:1–31:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2024), <https://doi.org/10.4230/LIPIcs.ITP.2024.31>
19. Ridge, T.: Simple, functional, sound and complete parsing for all context-free grammars. In: Jouannaud, J., Shao, Z. (eds.) Certified Programs and Proofs - First International Conference, CPP 2011. pp. 103–118. Lecture Notes in Computer Science, Springer (2011), https://doi.org/10.1007/978-3-642-25379-9_10

20. Scott, E.: SPPF-style parsing from Earley recognisers. *Electronic Notes in Theoretical Computer Science* **203**(2), 53–67 (Apr 2008). <https://doi.org/10.1016/j.entcs.2008.03.044>
21. Tomita, M.: *Efficient Parsing for Natural Language: A Fast Algorithm for Practical Systems*. Kluwer Academic Publishers (1985)

A Set Operations on type *list*

The definitions in this and the next appendix are mostly self-explanatory.

$$\begin{aligned} \text{filter } P \ [] &= [] \\ \text{filter } P \ (x \# xs) &= (\text{if } P \ x \text{ then } x \# \text{filter } P \ xs \text{ else } \text{filter } P \ xs) \\ \\ \text{fold } f \ [] \ s &= s \\ \text{fold } f \ (x \# xs) \ s &= \text{fold } f \ xs \ (f \ x \ s) \\ \\ \text{isin_list} \ [] \ a &= \text{False} \\ \text{isin_list} \ (x \# xs) \ a &= (\text{if } x = a \text{ then } \text{True} \text{ else } \text{isin_list} \ xs \ a) \\ \\ a \#_L \ xs &= (\text{if } \text{isin_list} \ xs \ a \text{ then } xs \text{ else } a \# xs) \\ \\ (\cup_L) &= \text{fold} \ (\#_L) \\ \\ xs -_L \ ys &= \text{filter} \ (\text{Not} \circ \text{isin_list} \ ys) \ xs \end{aligned}$$

B Set Operations on type *item_list*

$$\begin{aligned} \text{isin}_{IL} \ (IL \ as \ fs) \ x &= \text{isin_list} \ (fs \ ! \ \text{from } x) \ x \\ \\ \text{empty}_{IL} \ k &= IL \ [] \ (\text{replicate} \ (k + 1) \ []) \\ \\ x \#_{IL} \ IL \ as \ fs &= \\ &(\text{if } \text{isin}_{IL} \ (IL \ as \ fs) \ x \text{ then } IL \ as \ fs \\ &\text{else } IL \ (x \# \ as) \ (fs[\text{from } x := x \# \ fs \ ! \ \text{from } x])) \\ \\ [] \cup_{IL} \ il &= il \\ a \# \ as \cup_{IL} \ il &= a \#_{IL} \ as \cup_{IL} \ il \\ \\ IL_list \ k \ as &= as \cup_{IL} \ \text{empty}_{IL} \ k \\ \\ \text{diff}_{LIL} \ k \ [] \ il &= \text{empty}_{IL} \ k \\ \text{diff}_{LIL} \ k \ (a \# \ as) \ il &= \\ &(\text{if } \neg \text{isin}_{IL} \ il \ a \text{ then } a \#_{IL} \ \text{diff}_{LIL} \ k \ as \ il \text{ else } \text{diff}_{LIL} \ k \ as \ il) \\ \\ il_1 -_{IL} \ il_2 &= \text{diff}_{LIL} \ (|\text{froms } il_1| - 1) \ (\text{list } il_1) \ il_2 \end{aligned}$$

Note that

$empty_{IL}$ initializes the *froms* list with a list that is sufficiently large for the items that are expected to go into it. The required size can be determined statically in our setting. Function *replicate* creates a list of that size where every element is $\square$.

$(\cup_{IL})$ takes as its first argument a list.

$diff_{LIL}$ takes as a first argument the size of the required *froms* list and as the second argument a list.